FAKULTA INFORMATIKY A STATISTIKY

Ing. Karel Šafr, Ph.D.

Katedra statistiky a pravděpodobnosti

Python v pohybu

Od prvních příkazů k vlastní hře

4. listopadu 2026, 9:00–15:00, SB 321

Vysoká škola ekonomická v Praze

Pygame-ce pro náš workshop

Knihovna pro hry, simulace a interaktivní vizualizace v Pythonu.

Herní smyčku a pravidla píšeme přímo v Pythonu.

Po spuštění vidíme, jak se změny v kódu projeví ve hře.

Ukázky postupně přidávají pohyb, kolize, grafiku a práci s daty.

Web projektu: https://pyga.me/

Dokumentace: https://pyga.me/docs/

Proč vzniklo pygame-ce

V únoru 2023 část hlavních vývojářů pygame založila pygame-ce (Community Edition).

Podle jejich oznámení jim problémy v řízení původního projektu bránily pokračovat v jeho vývoji.

Fork převzal otevřený zdrojový kód a navázal na jeho rozhraní. Dál se vyvíjí jako samostatný projekt.

Cíle: častější vydání, průběžné opravy a více lidí zapojených do rozhodování o projektu.

Instalujeme pygame-ce. V kódu zůstává import pygame.

Jak číst dokumentaci

https://pyga.me/docs/

Vybereme modul a funkci. Sledujeme argumenty, návratovou hodnotu a popis chování.

```
rotate(surface, angle) -> Surface
```

V modulu pygame.transform: surface je zdrojový obrázek.
angle je úhel ve stupních, kladně proti směru hodinových ručiček.
Výsledkem je nový obrázek; zdroj zůstává stejný.

```
print(pygame.version.ver)
```

Vypíše nainstalovanou verzi. Online dokumentace může popisovat novější vývojovou verzi; sledujeme poznámky „New in“ a „Changed in“.

Co poskytuje pygame-ce

Grafika	Obrázky, kreslení tvarů a transformace.
Zvuk	Zvukové efekty a hudba.
Vstupy	Události klávesnice, myši a dalších zařízení.
Čas a kolize	Měření času, obdélníky a pomocné kolizní funkce.

Rychlost, tření, odrazy a herní pravidla určujeme vlastním kódem.

Další nástroje pro hry

pyglet	Python knihovna pro okno, grafiku, zvuk a vstupy.
Arcade	Python framework zaměřený na 2D hry.
Panda3D	3D engine s rozhraním pro Python a C++.
Godot	Samostatný editor a engine, například s jazykem GDScript.

Dnes pracujeme s pygame-ce a vlastní herní smyčkou.

Instalace a první spuštění

V terminálu nainstalujeme balíček pygame-ce:

```
python -m pip install pygame-ce
```

V programu se knihovna stále importuje pod názvem pygame:

```
import pygame
```

Otevřete 01_hello_world/hello_world.py a spusťte jej.

Ověříme okno, pohyblivý text a zavření křížkem.

V jednom prostředí instalujeme jen pygame, nebo pygame-ce.
Máte-li pygame, nejprve: python -m pip uninstall pygame

Knihovna, GUI, API a engine

pygame-ce, pyglet	Knihovny pro okno, grafiku a vstupy v Pythonu.
PyQt6, Kivy	Nástroje pro tvorbu uživatelského rozhraní.
OpenGL, Vulkan	Grafická API, ke kterým lze přistupovat z různých jazyků.
Godot	Samostatný herní engine s editorem a vlastními nástroji.

Tyto nástroje řeší různé části tvorby aplikace.

Proč hrají lidé hry?

Několik námětů k diskusi:

Zkoušet rozhodnutí a sledovat jejich důsledky.

Měnit prostředí a jeho pravidla.

Něco vytvářet a pozorovat výsledek.

Objevovat, co systém dovolí.

Pygame-ce a naše prostředí

Pygame-ce využívá knihovnu SDL.

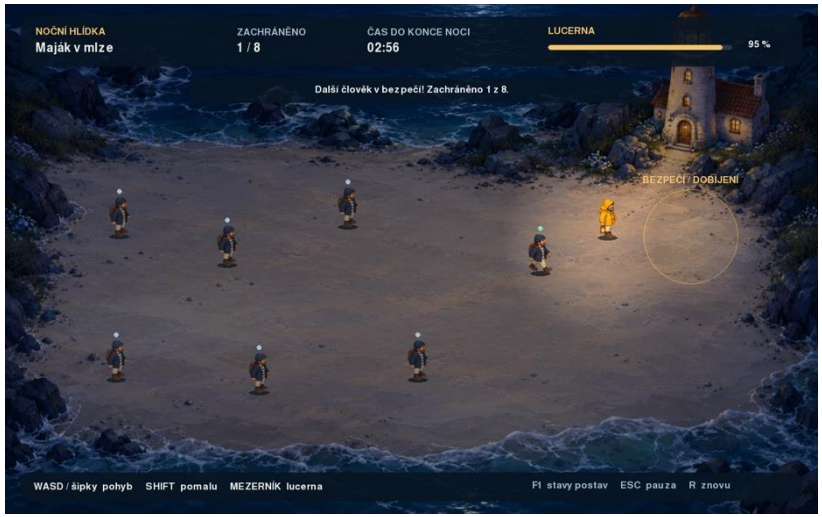
Otevírá okno a pracuje s obrázky i zvukem.

Zpracovává klávesnici, myš a další vstupy.

V tomto workshopu spouštíme místní programy .py na notebooku.

Balení aplikace pro telefon je samostatné téma.

Ukázky z tohoto workshopu



Maják v mlze

Světlo lucerny a záchrana trosečníků.



Ozvěny času

Záznam pohybu a spolupráce s ozvěnou.

Grafické podklady: OpenGameArt

OpenGameArt.org nabízí obrázky, animace a zvuky pro herní projekty.

U konkrétního balíčku sledujeme licenci a případné uvedení autora.



opengameart.org

Moduly pygame-ce

`display, draw`	Okno a kreslení.
`event, key, mouse`	Události a stav vstupů.
`time`	Časování a omezení snímkové frekvence.
`transform`	Změna velikosti, otočení a překlopení obrázků.
`joystick`	Gamepady a joysticky.

Další moduly a příklady jsou v dokumentaci.

display: okno a zobrazení

```
screen = pygame.display.set_mode((800, 600))
```

Vytvoří okno a vrátí zobrazovací Surface.

```
pygame.display.set_caption("Moje hra")
```

Nastaví titulek okna.

```
pygame.display.flip()
```

Zobrazí celý připravený snímek.

```
pygame.display.update()
```

Aktualizuje celý softwarový displej. update(rects) jen dané oblasti.
V režimu OPENGL se update() nepoužívá.

draw: kreslení tvarů

Příklady volání: screen je cílová Surface, color je barva.

```
pygame.draw.line(screen, color, (20, 20), (200, 80))
```

Úsečka mezi dvěma body; výchozí šířka 1 pixel.

```
pygame.draw.circle(screen, color, (100, 100), 30)
```

Vyplněný kruh: střed (100, 100), poloměr 30.

```
pygame.draw.rect(screen, color, (20, 20, 80, 40))
```

Vyplněný obdélník: x, y, šířka, výška.

```
pygame.draw.polygon(screen, color, [(20, 20),
                    (100, 20), (60, 80)])
```

Vyplněný mnohoúhelník; všechny funkce vracejí Rect změněné oblasti.

event: fronta událostí

Událost nese typ a případně data: například event.pos.

```
events = pygame.event.get()
```

Vrátí seznam čekajících událostí a odebere je z fronty.

```
event = pygame.event.poll()
```

Vyzvedne jednu událost. Nečeká: při prázdné frontě vrátí událost s typem pygame.NOEVENT.

```
pygame.event.post(event)
```

Vloží objekt Event do fronty; vrací True při vložení. Blokovaný typ se nevloží a vrátí False.

key: stav klávesnice

```
keys = pygame.key.get_pressed()
```

Stav čteme podle konstanty klávesy: keys[pygame.K_LEFT].
Krátký stisk mezi snímky zachytí spolehlivěji událost KEYDOWN.

```
pygame.key.set_repeat(400, 50)
```

Při držení klávesy opakuje KEYDOWN: po 400 ms každých 50 ms.
set_repeat() bez argumentů opakování vypne.

```
mods = pygame.key.get_mods()
```

Bitová maska modifikátorů: například mods & pygame.KMOD_SHIFT.

Stav kláves čteme po zpracování událostí, například event.get().

Čas v Pygame

```
clock = pygame.time.Clock()
```

Hodiny vytvoříme jednou před hlavní smyčkou.

```
clock.tick(60)
```

Omezí smyčku na nejvýše 60 FPS a vrátí milisekundy od předchozího tick().

```
pygame.time.get_ticks()
```

Vrací milisekundy od inicializace Pygame.

pygame.time.delay(ms) pozastaví běh programu.

mouse: kurzor a tlačítka

```
x, y = pygame.mouse.get_pos()
```

Poloha v pixelech vzhledem k levému hornímu rohu okna.

```
left, middle, right = pygame.mouse.get_pressed()
```

Ve výchozím nastavení vrátí stav tří tlačítek myši.
Čteme jej po zpracování událostí.

```
pygame.mouse.set_visible(False)
```

False kurzor skryje, True zobrazí.
Vrací předchozí stav viditelnosti.

transform: změny obrázku

```
scaled = pygame.transform.scale(image, (150, 100))
```

Nová velikost v pixelech; rychlé škálování bez vyhlazení.

```
rotated = pygame.transform.rotate(image, 30)
```

Otočení o 30 stupňů proti směru hodinových ručiček.
Záporný úhel otáčí po směru; rozměry se mohou změnit.

```
flipped = pygame.transform.flip(image, True, False)
```

První boolean: vodorovně. Druhý: svisle.

Tato volání vracejí nový Surface. Původní image nemění.

Sprite a skupiny objektů

Sprite	Základní třída pro objekt s vlastním chováním.
Group	Skupina objektů pro společnou aktualizaci a kreslení.
Kolizní funkce	Pomáhají zjistit, které objekty se překrývají.

Vlastní třída může ze Sprite dědit a přidat další stav a metody.

Objekty a moduly pro grafiku a zvuk

`pygame.Surface`	Třída pro obrázek uložený v paměti.
`pygame.Rect`	Třída pro obdélník, polohu a rozměry.
`pygame.mixer.Sound`	Třída pro načtený zvukový efekt.
`pygame.mixer.music`	Modul pro streamované přehrávání hudby.
`pygame.font`	Modul pro fonty a vykreslení textu.

FPS

FPS = počet vykreslených snímků za sekundu.

Pohyb vzniká tím, že v každém snímku zobrazíme objekt na trochu jiné pozici.

```
clock = pygame.time.Clock()     # před smyčkou
clock.tick(60)                  # ve smyčce
```

60 je horní limit. Když výpočet trvá déle, skutečné FPS klesne.

V první ukázce měníme obě souřadnice o 1 pixel za snímek.
Při přibližně 60 FPS jde asi o 60 pixelů za sekundu v každé ose.

Herní smyčka

1. EVENT – zpracujeme události

2. UPDATE – změníme stav hry

3. RENDER – vykreslíme a zobrazíme snímek

První ukázka: posun za snímek + omezení na nejvýše 60 FPS.

Pozdější možnost: rychlost v pixelech za sekundu × uplynulý čas dt.
Samotný limit FPS nezajistí stejnou rychlost na pomalém počítači.

Šablona – inicializace

```
import pygame
pygame.init()

screen = pygame.display.set_mode((800,600))
pygame.display.set_caption("Ahoj svete!")

clock = pygame.time.Clock()
myfont = pygame.font.Font(None, 50)

running = True
i = 1
```

Okno, hodiny a font připravíme jednou před hlavní smyčkou.

Šablona – události

```
while running:
    # Event
    for event in pygame.event.get():
        print(event)
        if event.type == pygame.QUIT:
            running = False
```

get() vyzvedne čekající události z fronty a zároveň je odebere.
print(event) nám jen umožní sledovat, co přišlo.

Zavření okna vyšle QUIT. Hodnota running = False ukončí smyčku.

Šablona – aktualizace

```
i+=1
# Update
textik = myfont.render(
    f"Ahoj Svete!{i}", True, (250,80,100)
)
```

Zvětšíme i a vytvoříme nový obrázek textu s aktuálním číslem.
Stejné i později použijeme také jako souřadnice.

Font už existuje; v každém snímku se mění pouze vykreslený text.

Dlouhý výpočet nebo kreslení sníží FPS. tick() zde neřeší pravidla hry.

Šablona – vykreslení a zobrazení

```
    # Render
    screen.fill((0,0,0))
    screen.blit(textik,(0+i,0+i))
    pygame.display.update()
    clock.tick(60)

pygame.quit()
```

fill vyplní plochu; blit do ní vloží text. update() zobrazí snímek.
Pro zobrazení celého snímku lze použít také display.flip().

Zobrazení následuje po kreslení; potom zde omezíme FPS.
pygame.quit() je až za smyčkou a uvolní prostředky Pygame.

Herní smyčka – pseudokód

```
Připravit knihovnu, okno, hodiny a font
Nastavit running a počáteční hodnotu i

Dokud running platí:
    Vyzvednout a zpracovat události
    Při zavření okna nastavit running na False
    Změnit hodnoty a připravit text
    Vyčistit plochu a vykreslit nový stav
    Zobrazit snímek
    Omezit smyčku na nejvýše 60 FPS

Uvolnit prostředky Pygame
```

Toto je popis postupu, nikoliv Python k opsání.
První ukázka končí křížkem; obsluhu klávesy ESC lze přidat později.

01_Hello world: příprava

```
import pygame
pygame.init()
BLACK = (0,0,0)
PURPLE = (150, 10, 100)
screen = pygame.display.set_mode(
    (800,600)
)
pygame.display.set_caption("Ahoj svete!")
screen.fill(PURPLE)
clock = pygame.time.Clock()
myfont = pygame.font.Font(None, 50)
running = True
i = 1
```

01_Hello world: smyčka

```
# cyklus udrzujici okno v chodu
while running:
    # Event
    for event in pygame.event.get():
        print(event)
        if event.type == pygame.QUIT:
            running = False
    i+=1
    # Update
    textik = myfont.render(
        f"Ahoj Svete!{i}",
        True, (250,80,100)
    )
    # Render
    screen.fill((0,0,0))
    screen.blit(textik,(0+i,0+i))
    pygame.display.update()
    clock.tick(60)
pygame.quit()
```

Hello world – co sledovat

Příprava a běh

pygame.init() připraví knihovnu.
set_mode() vytvoří okno.
Hodiny a font vytvoříme jednou před smyčkou.

V každém snímku změníme i.
Vytvoříme nový obrázek textu a vykreslíme jej na (i, i).
print(event) ukazuje události.

Po zavření okna skončí smyčka.
pygame.quit() uvolní prostředky.
Samo o sobě neukončuje celý Python.

Otázky k pokusu

Proč nevidíme fialové pozadí?
Před prvním zobrazením je přepíše černá výplň uvnitř smyčky.

Kam zmizí text?
Hodnota i dál roste. Při asi 60 FPS text přibližně za 10 sekund vyjede z okna.

Co změní zápis (0+i, 0+i) na (i, i)?
Vyzkoušejte také jinou barvu, velikost písma nebo hodnotu v clock.tick().

02_ball

Živá ukázka v Pythonu

`02_ball/ball.py`

Vykreslení snímku

V každém průchodu smyčkou vytvoříme celý obraz:

`screen.fill(black)`	Vyčistí plochu.
`screen.blit(ball, ballrect)`	Vloží míček na jeho pozici.
`pygame.display.flip()`	Zobrazí hotový snímek.

Dvojité bufferování a zobrazení snímku

Dva buffery střídají přípravu a zobrazení obrazu.
Samotné flip() tento režim nezaručuje; záleží na displeji.

U OPENGL provede flip() výměnu GL bufferů. U softwarového displeje lze pomocí update(obdélníky) obnovit jen části obrazu.

Kreslení může být náročné. Co program brzdí, zjistíme měřením.
Pro tuto ukázku stačí překreslit celé okno.

Sprite a kolize

Sprite: data a chování objektu

V následující ukázce použijeme sprite pro pohybující se kostku.
Její obrázek, polohu a pohyb spojíme do jedné třídy.

Test kolize a reakce hry

Pygame nabízí funkce pro zjištění překryvu objektů.
Co při překryvu uděláme, určují pravidla napsaná v našem kódu.

Nejprve si ukážeme jednu kostku a skupinu spritů.
Ke kolizím se vrátíme ve třetí variantě.

Sprite v naší ukázce

Data

`self.image`	Obrázek kostky, který se bude kreslit.
`self.rect`	Poloha a obdélníkový rozsah kostky.

Chování

```
def update(self):
```

Zde měníme polohu a řešíme přechod okrajem.

V našich ukázkách odvozujeme třídu kostka od pygame.sprite.Sprite.
Použití spritů je způsob organizace kódu, není povinné pro každou hru.

Základní grafické tvary

Přehled funkcí pro kreslení do plochy Surface:

`pygame.draw.ellipse`	elipsa
`pygame.draw.circle`	kruh
`pygame.draw.line`	úsečka
`pygame.draw.rect`	obdélník

Jde o názvy funkcí, nikoliv úplné příkazy ke spuštění.
Při volání doplníme cílovou plochu, barvu a rozměry či souřadnice.

Skupina: update a draw

Před smyčkou vytvoříme skupinu a vložíme do ní kostku:

```
my_sprites = pygame.sprite.Group()
kosticka = kostka()
my_sprites.add(kosticka)
```

V každém snímku

`my_sprites.update()`	Zavolá update() každého člena.
`screen.fill(BLACK)`	Vyčistí plochu.
`my_sprites.draw(screen)`	Vykreslí image na pozici rect.
`pygame.display.flip()`	Zobrazí dokončený snímek.

Group.draw() kreslí do předané plochy screen.
Samo aktualizaci ani zobrazení snímku nespouští.

Sprite: image a rect

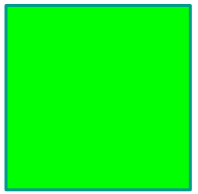
Pro vykreslení pomocí Group.draw() připravíme image a rect.

```
self.image = pygame.Surface((50,50))
self.image.fill(GREEN)
```

image obsahuje pixely obrázku.

```
self.rect = self.image.get_rect()
```

rect popisuje polohu a obdélníkový rozsah.
Změna jeho velikosti sama nezvětší obrázek.



50 × 50 px

Vlastní update() přidáme pro pohyb. Kolizní metody zde nepotřebujeme.

Deklarace kostky

```
class kostka(pygame.sprite.Sprite):
    def __init__(self):
        pygame.sprite.Sprite.__init__(self)
        self.image = pygame.Surface((50,50))
        self.image.fill(GREEN)
        self.rect = self.image.get_rect()
        self.rect.center = (WIDTH/2,HEIGHT/2)

my_sprites = pygame.sprite.Group()
kosticka = kostka()
my_sprites.add(kosticka)
```

Výřez ze skriptu: konstruktor a přidání instance do skupiny.
Metoda update() určuje pohyb a přechod okrajem.

Pohyb v metodě update

```
my_sprites.update()
```

Náš program tím zavolá update() každé kostky ve skupině.

Ve skriptu: výřez metody

```
def update(self):
    self.rect.x -= 5
    self.rect.y += 1
```

Pokus: opačný směr

```
def update(self):
    self.rect.x += 5
    self.rect.y -= 1
```

Osa x roste doprava, osa y dolů.
Ve skriptu se tedy kostka posouvá vlevo dolů.

Jde o posun za snímek. Hodiny omezují běh na nejvýše 45 FPS.

03_kosticky: první varianta

Živá ukázka v Pythonu

`03_kosticky/template_kostka.py`

Sprite: základ a rozšíření

Atribut	Význam v naší kostce
image	Obrázek, který skupina vykreslí. Typ pygame.Surface.
rect	Poloha a obdélníkový rozsah. Typ pygame.Rect.

Pro pozdější ukázky

mask: přesnější test překryvu podle pixelů.

dirty, visible, blendmode: vlastnosti DirtySprite,
využívané odpovídajícími skupinami, například LayeredDirty.

layer: pořadí vykreslení ve vrstvené skupině.

Naše běžná Group uvedené vlastnosti DirtySprite při kreslení nepoužívá.

Pohyb obdélníku Rect

Zápis	Výsledek	Původní Rect
novy = rect.move(dx, dy)	Vrátí nový Rect.	Zůstává stejný.
rect.move_ip(dx, dy)	Vrátí None.	Změní se.
rect.x += dx rect.y += dy	Přiřazení do souřadnic.	Změní se.

Zde pracujeme s celočíselnými posuny. Rect ukládá celé souřadnice.

U desetinných posunů se převod na celá čísla může lišit.
Tyto zápisy proto nejsou obecně zaměnitelné.

Nový Rect, nebo změna původního?

Nový obdélník

```
rect = pygame.Rect(10,20,50,50)
novy = rect.move(5,0)
print(rect.x)   # 10
print(novy.x)   # 15
```

Změna původního

```
rect = pygame.Rect(10,20,50,50)
vysledek = rect.move_ip(5,0)
print(rect.x)     # 15
print(vysledek)   # None
```

move() využijeme například pro zkoušku budoucí polohy.
move_ip() nebo přímá změna souřadnic mění stávající Rect.

Malé desetinné kroky sčítáme v samostatné poloze typu float.
Do Rect kopírujeme až výslednou souřadnici; příklad je u Hráče v ploše.

Sprite: metody a skupiny

Metoda	Význam
update()	Naše chování objektu, volané z Group.update().
add(skupina)	Přidá sprite do skupiny.
remove(skupina)	Odebere jej ze zadané skupiny.
kill()	Odebere jej ze všech skupin. Objekt může dále existovat.
alive()	Vrací, zda je členem alespoň jedné skupiny.
groups()	Vrátí seznam jeho skupin.

my_sprites.update(dt) předá dt do update(self, dt) každého spritu.
Naše metoda jej musí použít: posun = rychlost × dt.
Posun přičítáme k desetinné poloze; z ní nastavíme Rect.

Průběžné ovládání klávesnicí

Držení šipky zjišťujeme během každého update().

Výřez metody update(): vodorovný pohyb

```
self.speedx = 0
self.speedy = 0
keystate = pygame.key.get_pressed()
if keystate[pygame.K_LEFT]:
    self.speedx -= self.delta
if keystate[pygame.K_RIGHT]:
    self.speedx += self.delta
```

Na začátku aktualizace obě rychlosti vynulujeme.

Samostatné if dovolí současné klávesy. Levá a pravá se vyruší.

get_pressed() vrací aktuální stav kláves, ne události z fronty.

Group.update() zavolá update() všech kostek. Každá čte stejnou klávesnici, takže se pohybují společně. Obdobně fungují šipky nahoru a dolů.

Událost myši: nová kostka

Nová kostka vzniká při uvolnění libovolného tlačítka myši.

Výřez z obsluhy událostí v hlavní smyčce:

```
for event in pygame.event.get():
    if event.type == pygame.MOUSEBUTTONUP:
        k = kostka(event.pos[0],event.pos[1])
        my_sprites.add(k)
```

event.pos určuje počáteční střed nové kostky.
my_sprites.add(k) ji zařadí do stejné skupiny jako ostatní.

Držení šipky čteme průběžně; uvolnění tlačítka zpracujeme jednou jako událost. Při následujícím update() se už i nová kostka může posunout.

03_kosticky: druhá a třetí varianta

`template_kostka2.py`

Ovládání šipkami a přidávání kostek myší.

`template_kostka3.py`

Další krok: reakce na překryv při uvolnění tlačítka myši.

Kolize: porovnání spritů

Jeden sprite proti skupině

```
zasahy = pygame.sprite.spritecollide(k, my_sprites, False)
```

Vrací seznam spritů, jejichž rect se překrývá s k.rect.
False pouze testuje. True nalezené sprity odebere ze všech jejich skupin.

Dvě skupiny

```
zasahy = pygame.sprite.groupcollide(
    skupina1, skupina2, False, False)
```

Vrací slovník: sprite z první skupiny → seznam zásahů ve druhé.
Obě False ponechají členství spritů beze změny.

Zde jde o překryv obdélníků. Pouhý dotyk jejich hran nestačí.

Kostka 3: přidání nebo odebrání

Výřez obsluhy událostí v template_kostka3.py:

```
if event.type == pygame.MOUSEBUTTONUP:
    k = kostka(event.pos[0],event.pos[1])
    if pygame.sprite.spritecollide(k, my_sprites,True):
        k.kill()
    else:
        my_sprites.add(k)
```

Bez překryvu se nová kostka přidá. Při překryvu se odstraní všechny zasažené staré kostky a nová se nepřidá. Mazání starých provádí True.

Test probíhá při uvolnění tlačítka myši. Při samotném pohybu se kolize nekontrolují. Význam ponechaného k.kill() si ukážeme dále.

Co dělá Sprite.kill()

kill() odebere sprite ze všech skupin, do kterých právě patří.

Pokus s kostkou ve skupině

```
kosticka = kostka(200, 200)
my_sprites.add(kosticka)
kosticka.kill()
```

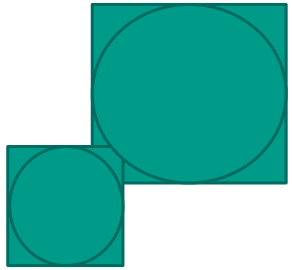
Skupina už tuto kostku nevykreslí ani neaktualizuje. Objekt dál existuje v proměnné kosticka; můžeme jej do skupiny znovu přidat.

k.kill() ve třetí variantě

Nová k ještě není v žádné skupině, takže ji toto volání nemá odkud odebrat. Řádek ponecháváme jako ukázku použití kill().

Rozšíření: kruhová kolize

Obdélníkový test může zachytit i prázdné rohy obrázku.
Kruhový test použije střed rect a atribut radius.



Náhrada tvorby obrázku v konstruktoru každé kostky:

```
self.image = pygame.Surface((50,50), pygame.SRCALPHA)
self.radius = 25
pygame.draw.circle(self.image, GREEN,
                   self.image.get_rect().center, self.radius)
self.rect = self.image.get_rect()
self.rect.center = (x,y)

zasahy = pygame.sprite.spritecollide(
    k, my_sprites, False, pygame.sprite.collide_circle)
```

Samostatný pokus: místní střed image, poloměr 25 pro všechny kostky.
False pouze testuje. Soubor kostka3 nadále používá obdélníky.

Kostka 3: pokusy v editoru

`03_kosticky/template_kostka3.py`

Volné místo

Uvolnění tlačítka myši přidá kostku.

Překryv jedné nebo více kostek

Zasažené kostky zmizí a nová se nepřidá.

Vyzkoušej i pouhý dotyk hran: ten nestačí. Testuje se celá plocha nové kostky 50 × 50, takže zásah může nastat i s kurzorem vedle staré.

Obrázek na pozadí

Doplňkový příklad s vlastním souborem nasepozadi.png.
Hráč v ploše místo obrázku kreslí mřížku.

Jednou před smyčkou

```
screen = pygame.display.set_mode((800,600))
background = pygame.image.load("nasepozadi.png").convert()
```

V každém snímku

```
screen.fill((0,0,0))
screen.blit(background, (0,0))
```

Obrázek je Surface; Rect popisuje polohu a rozměry.
Konverzi provádíme po vytvoření okna.
Alfa kanál průhledného PNG zachováme pomocí convert_alpha().

04_hrac_v_plose

`04_hrac_v_plose/main.py`

Světové a obrazovkové souřadnice

Krok 1,5 px za snímek; limit 60 FPS.
Při dosažených 60 FPS jde v jedné ose přibližně o 90 px/s.

Kamera a hráč v ploše

obrazovka = svět − kamera

Hráč (1300, 1100) − kamera (900, 800) = obrazovka (400, 300).

Výřez: výpočet kamery v ose x

```
camera_x = max(0, min(
    player_x - WIDTH // 2, MAP_WIDTH - WIDTH))
```

Kamera sleduje hráče, dokud nenarazí na hranici mapy.
U kraje se zastaví a hráč už není uprostřed. Osa y funguje stejně.

Mapa je alespoň tak velká jako okno. Od souřadnic hráče i dlaždic
při kreslení odečítáme kameru. Meze pohybu počítají i s poloměrem hráče.

Desetinná poloha a pixely

Rect uchovává celé souřadnice; malé přírůstky v něm mohou zaniknout. Desetinnou polohu proto udržujeme zvlášť mezi jednotlivými snímky.

Pokus se spritem: poloha zvlášť, Rect pro vykreslení

```
# před smyčkou
x = float(rect.x)

# v každém snímku
x += 0.25
rect.x = round(x)
```

Čtyři kroky po 0,25 px dají posun 1 px; stejně i v opačném směru. Desetiny zůstávají v x. Pro osu y platí stejný postup. Hráč v ploše už uchovává desetiny v player_x a player_y.

05_Template

`05_template/template.py`

Po spuštění se otevře prázdné černé okno.

Připravená místa postupně doplníme o grafiku,
vlastní sprity a herní pravidla.

Šablona hry

Konstanty pro barvy, rozměry a FPS jsou nahoře pro snadnou úpravu.

Příprava před smyčkou

```
pygame.init()
pygame.mixer.init()
```

Okno: set_mode() a titulek.
Grafika a třídy: místo k doplnění.
Hodiny a prázdná skupina spritů.

V každém snímku

Event: vstup a ukončení
Update: my_sprites.update()
Render: výplň, draw(), flip()

clock.tick(FPS): nejvýše 45 FPS.
Pomalejší snímky nezrychlí.

První import tohoto souboru spustí i vytvoření okna a hlavní smyčku.

06_CAR

`06_CAR/car.py`

Jednoduchý model jízdy s vektorovou rychlostí.

Nahoru / dolů: plyn a opačný tah.
Vlevo / vpravo: zatáčení při pohybu.

Přesouváme celou složku včetně obrázků PNG.
Cesty k obrázkům se odvozují od umístění skriptu (__file__).

Pohyb auta a mantinely

Vector2 uchovává desetinnou polohu a rychlost.
Rect určuje celočíselnou polohu obrázku a slouží k testu překryvu.

Pořadí v update()

Vstup a úhlově závislé tření.
Posun, potom omezení rychlosti.
Rotace a ohraničení obrazovky.
Test kolize s mantinely.

Odraz násobkem −1,5

Obrátí celý směr pohybu.
Velikost rychlosti zvětší 1,5krát.
Například (4, 0) změní na (−6, 0).
Sám neodstraní překryv se stěnou.

```
dt = clock.tick(FPS) / 10
```

tick() vrací milisekundy. Dělení 10 nedává sekundy.
Posun a tření jsou za snímek. Výsledná jízda tedy závisí i na FPS.

Rotace původního obrázku

rotozoom() vrací nový otočený a filtrovaný Surface.

Vždy z original_image

Zdrojové pixely zůstávají stejné.
Použijeme aktuální úhel auta.
Rozměry výsledku se mění s úhlem.

Opakovaně z self.image

Další rotace mění předchozí výsledek.
Zhoršení kvality se může hromadit.
Obalový obdélník může narůstat.

Obě varianty mohou volat transformaci jednou za snímek.
Rostoucí plocha obrázku zvyšuje práci při dalším přepočtu.

Z otočeného obrázku vytvoříme nový Rect se zachovaným středem.

Rotace: car2.py a car.py

car2.py: na začátku update()

```
previous_angle = self.angle
```

Potom proběhne ovládání a změna self.angle.



car2.py

Záměrně chybně: otočení předchozího výsledku

```
delta_angle = self.angle - previous_angle
self.image = pygame.transform.rotozoom(
    self.image, -delta_angle, 1)
```

Správně: z původního obrázku v car.py

```
self.image = pygame.transform.rotozoom(
    self.original_image, -self.angle, 1)
self.rect = self.image.get_rect(center=self.rect.center)
```

Jednorázová animace

`07_Animace/Animace.py`

Šest obrázků z jednoho PNG vyřízneme pomocí subsurface.



Limit 60 FPS, animace 8/s

```
dt = clock.tick(FPS) / 1000
```

dt: čas od minulého tick() v sekundách.
Obrázek se střídá po 0,125 s.

ENTER spouští animaci

Za běhu další stisk ignorujeme.
Na konci zůstane poslední obrázek.
Dalším ENTER začneme od prvního.

Při delším dt smyčka while dožene uplynulé intervaly.
Mezilehlé obrázky pak mohou být při vykreslení přeskočené.

Grafika: tvar a průhlednost

Obrázek spritu uložíme do self.image.

```
image = pygame.transform.scale(image, (150, 100))
```

Z 75 × 50 na 150 × 100: poměr stran zůstává stejný.

```
image = pygame.transform.flip(image, True, False)
```

První boolean: vodorovně. Druhý: svisle.

Průhledné PNG

Alfa kanál zachová convert_alpha().

Barevný klíč

```
image.set_colorkey((0, 0, 0))
```

Skryje danou RGB barvu.
Příklad pro obrázek bez alfa kanálu.

Jeden obrázek, jedno načtení

Připravíme jej před smyčkou, po vytvoření okna.

```
image = pygame.image.load(
    ASSETS_PATH / "A.png").convert_alpha()
rect = image.get_rect(center=(400, 300))
```

Surface žije, dokud na něj něco odkazuje

Odkaz může držet sprite, proměnná nebo cache.
Zánik jednoho spritu při sdílení obrázek neuvolní.

Více stejných spritů může používat tentýž načtený Surface.

Změna kopie sdíleného obrázku

Funkce tint z naší ukázky upraví vlastní kopii.

```
def tint(surface: pygame.Surface,
         color: tuple[int, int, int]) -> pygame.Surface:
    copy = surface.copy()
    copy.fill(color, special_flags=pygame.BLEND_MULT)
    return copy

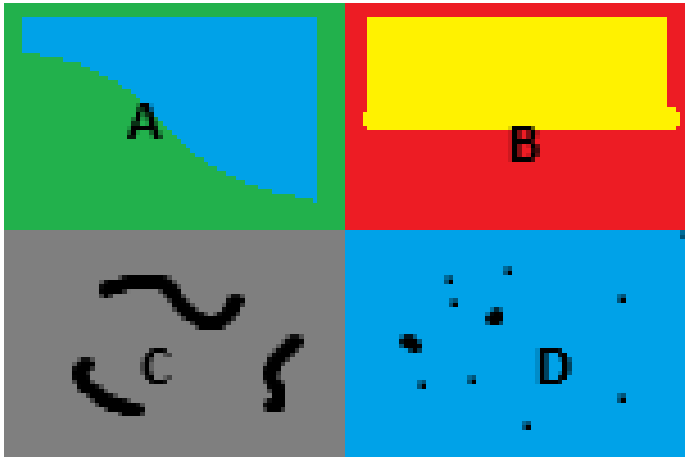
red_b_img = tint(b_img, (255, 100, 100))
```

fill() a zápis přes PixelArray mění pixely plochy na místě.

rotate() a rotozoom() vracejí nový Surface. Také scale(),
pokud mu nepředáme volitelný dest_surface.

Spritesheet: čtyři výřezy

sprite_sheet.png: 150 × 100 px, čtyři pole 75 × 50 px.



Pořadí výřezů

A, B, C, D
Nejprve řádek, potom další řádek.

```
TILE_W, TILE_H = 75, 50
frames = ResourceCache.sheet(
    "sprite_sheet.png",
    TILE_W, TILE_H)
```

subsurface sdílí pixely s rodičem.
Změna výřezu se projeví i v původním obrázku.

Kdy znovu přepočítat obrázek

Stejné rozměry: připravíme jednou

```
scaled = pygame.transform.scale(original, (150, 100))
```

Měnící se úhel: rotujeme z originálu

```
self.image = pygame.transform.rotozoom(
    self.original_image, -self.angle, 1)
self.rect = self.image.get_rect(center=self.rect.center)
```

Stejný výsledek znovu použijeme; měnící se úhel můžeme přepočítat každý snímek, jako u auta.

Předpočítání samo o sobě nezaručuje plynulé FPS.

Sdílená cache obrázků

Zjednodušená obdoba ResourceCache.image z ukázky.
Naše PNG má alfa kanál; colorkey při volání nepředáváme.

```
_res_cache = {}

def load_image(name, colorkey=None):
    if name not in _res_cache:
        img = pygame.image.load(ASSETS_PATH / name)
        img = img.convert_alpha()
        if colorkey is not None:
            img.set_colorkey(colorkey)
        _res_cache[name] = img
    return _res_cache[name]
```

Klíčem je pouze název souboru.

colorkey nastavujeme jen při prvním načtení.
Další volání stejného jména vrátí již uložený Surface.

Shrnutí práce s obrázky

1. Načteme jednou při přípravě objektu nebo scény.

2. Stejný neměněný Surface můžeme sdílet.

3. Před vlastní změnou sdílených pixelů vytvoříme kopii.

4. Transformujeme originál a opakované výsledky uložíme.

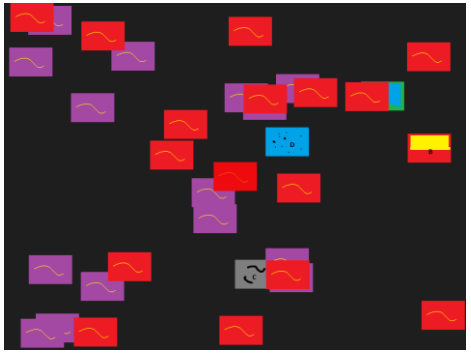
Cache omezuje opakované načítání; sdílení omezuje kopie.
Plynulost celé hry ověřujeme měřením.

08_Image: pohyb a vrstvy

08_Image/main.py

V konstruktoru, po vytvoření Rectu

```
self.x = float(self.rect.x)
self.y = float(self.rect.y)
```



V každém update(dt)

```
self.x += self.vx * dt
self.y += self.vy * dt
self.rect.topleft = (round(self.x), round(self.y))
```

self.x a self.y uchovávají desetinnou polohu mezi snímky.
Rect se z ní nastaví pro vykreslení a kolize; dt je v sekundách.

Vrstvy 0–3: pozadí, výřezy, A/B, tónovaná kopie B.

0x_SIM: model šíření nákazy

1 000 osob, na začátku jedna nakažená.

Jeden simulační krok

```
population.update()
```

Pohyb, uzdravení a úmrtí všech osob.

```
for individual in population:
    individual.try_to_infect(population)
```

Přenos při překryvu: pokus s pravděpodobností 0,1.

Potom statistiky a vykreslení osob i všech tří grafů.

Nově nakažený může šířit nákazu už v témže průchodu.

Časový krok a pravděpodobnost

Čas v této simulaci

```
FPS = 45
RECOVERY_TIME = FPS * 10
```

Uzdravení po 450 aktualizacích.

tick(45) omezuje rychlost.
Dosažených 45 FPS nezaručuje.

10 sekund jen při 45 krocích/s.

Opakovaný náhodný pokus

```
DEATH_PROBABILITY = 0.005
```

Za jeden infekční krok: 0,5 %.

Před uzdravením až 449 pokusů.

```
p = DEATH_PROBABILITY
celkem = 1 - (1 - p)**449
```

Před uzdravením: asi 89,47 %.

V kroku 450 se nejprve uzdraví; pokus o úmrtí už nenastane.

89,47 % platí jen pro tento model, ne pro covid-19.

Jak číst tři grafy

Stejný průběh simulace, tři pohledy na data.

Někdy nakažení



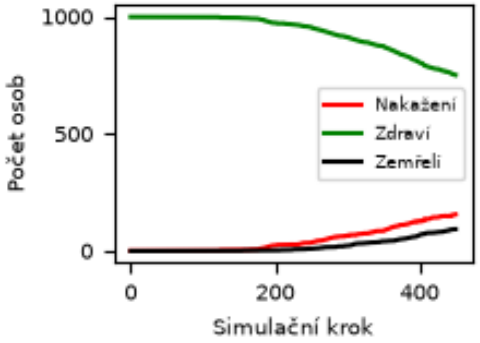
Podíl v procentech.
Kumulativní podíl neklesá.

Současné počty



Zdraví zahrnují zotavené.
„Nikdy nakažení“ jsou část zdravých.

Vývoj v krocích



Tři stavy populace.
Osa x: simulační krok.

První tři sloupce tvoří populaci. Čtvrtý se s nimi překrývá.

0x_SIM: vlastní experiment

0x_SIM/SIM.py

Potřebné knihovny nainstalujeme v terminálu:

```
python -m pip install matplotlib numpy
```

Malý experiment

1. Změňte v kódu TRANSMISSION_PROBABILITY.
2. Pro každou hodnotu spusťte simulaci několikrát.
3. Porovnejte grafy po stejném počtu kroků.

Náhoda mění výsledek. Parametry upravujeme mezi běhy.

Zvuk: načíst a spustit

Příprava před herní smyčkou

```
pygame.init()
pygame.mixer.init()


folder = path.dirname(__file__)

punch = pygame.mixer.Sound(
    path.join(folder, "sfx_punch.wav"))
```

Sound načte WAV jednou do paměti.

Při úderu

`punch.play()` Spustí zvuk na pozadí; herní smyčka pokračuje.

09_Fighter: pohyb, animace a úder

09_Fighter/fighter.py

Chůze a úder

← / →: posun a otočení postavy.
Držení SPACE: úder každý update.

Posun ±5 probíhá každý update.
Desetinné self.x uchováme;
Rect nastavíme přes round().

Zelený hitbox zásahu

Při úderu vznikne dočasný sprite.
Ukázka zatím nemá protivníka.

Časová značka v __init__

```
self.start = (
    pygame.time.get_ticks())
```

Obrázek má vlastní časovač

```
now = pygame.time.get_ticks()
if now - self.last_update > 25:
    self.frame = (
        (self.frame + 1) % 8)
    self.last_update = now
```

Rozdíl značek v update

```
if (pygame.time.get_ticks()
        - self.start > 250):
    self.kill()
```

Rozdíl značek měří uplynulé ms.
Ve 250 ms zůstává; po překročení
se při update odebere ze skupin.

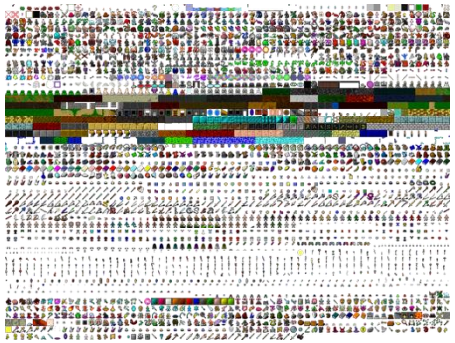
Dlaždicová mapa z jednoho atlasu

Okno 320 × 320 px; mapa 10 × 10 dlaždic po 32 × 32 px.

```
from pathlib import Path
sprite_sheet_path = Path(__file__).with_name(
    "DungeonCrawl_ProjectUtumnoTileset_0.png")
sprite_sheet = pygame.image.load(sprite_sheet_path)
```

__file__ je cesta k tomuto skriptu.
Atlas hledáme ve stejné složce.

Původní atlas 2048 × 1536 px



Atlas načteme jednou před smyčkou.

Výřez atlasu: indexy a pixely

get_tile(x, y): sloupec a řádek atlasu, počítané od nuly.

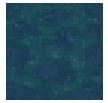
```
def get_tile(x, y):
    return sprite_sheet.subsurface(
        x * tile_size, y * tile_size,
        tile_size, tile_size)
```

Příklad trávy
Index (1, 13)
Začátek (32, 416) px

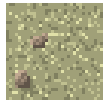
0 = tráva `grass_tile = get_tile(1, 13)`



1 = voda `water_tile = get_tile(19, 19)`



2 = poušť `desert_tile = get_tile(19, 13)`



Výběr v atlasu určí obrázek. Umístění v mapě určí matice.

Čísla mapy se mění na obrázek

tile_map[y][x]: nejprve řádek, potom sloupec.
enumerate přidá k hodnotám jejich indexy y a x.

```
def draw_map():
    for y, row in enumerate(tile_map):
        for x, tile in enumerate(row):
            if tile == 0:
                screen.blit(grass_tile,
                    (x * tile_size, y * tile_size))
            elif tile == 1:
                screen.blit(water_tile,
                    (x * tile_size, y * tile_size))
            elif tile == 2:
                screen.blit(desert_tile,
                    (x * tile_size, y * tile_size))
```

blit položí vybranou dlaždici na pozici (x × 32, y × 32).

Smyčka statické mapy

Clock vytvoříme jednou

```
clock = pygame.time.Clock()

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((0, 0, 0))
    draw_map()
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Události

Zavření okna ukončí smyčku přes QUIT.

Překreslení

fill: černé pozadí
draw_map: 100 polí
flip: hotový snímek

tick(60): horní limit rychlosti smyčky.

Co si z dlaždic odnést

1. Data oddělená od kreslení

tile_map uloží čísla. draw_map je vykreslí.

2. Mřížka se převádí na pixely

tile_map[y][x] → (x × 32, y × 32)

3. Výřezy sdílejí pixely atlasu

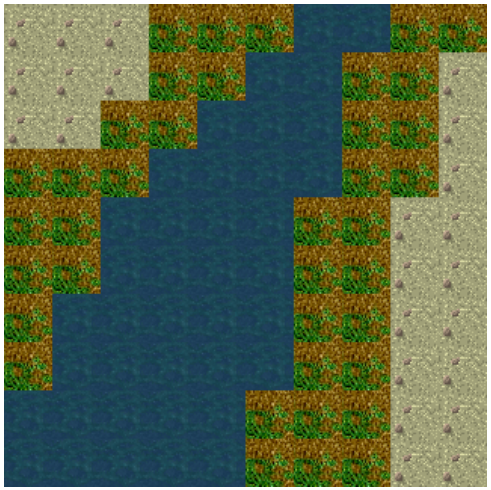
subsurface nevytváří nezávislou kopii pixelů.

Možné další pokusy (zatím nejsou v kódu)

Slovník dlaždic, convert_alpha(), předrenderovaná mapa.

10_tiles: změňte jednu buňku

10_tiles/main_tiles.py



Výchozí mapa: 10 × 10 polí.

Úkol v editoru

V prvním řádku tile_map změňte první 2 na 1.
Uložte soubor a znovu spusťte program.

Původní první řádek

```
[2, 2, 2, 0, 0, 0, 1, 1, 0, 0]
```

Po změně

```
[1, 2, 2, 0, 0, 0, 1, 1, 0, 0]
```

Vlevo nahoře se poušť změní na vodu.

Herní izometrie: pohled ve 2D

Souřadnice mřížky převádíme na šikmo zobrazená pole.

Poměr dlaždice 2:1

Šířka je dvojnásobek výšky.
Tady 64 × 32 px.

64 : 32 = 2 : 1

V této první ukázce

Mřížku 10 × 10 vytvoří dva cykly. Matice se neukládá.
Kreslíme zelené polygony a červený kruh.

Textury a posouvání mapy přijdou v další ukázce.

Z mřížky na obrazovku

grid_to_iso vrací horní vrchol dlaždice.

```
def grid_to_iso(x, y):
    screen_x = (x - y) * (TILE_WIDTH // 2) + SCREEN_WIDTH // 2
    screen_y = (x + y) * (TILE_HEIGHT // 2) + 50
    return screen_x, screen_y
```

Vodorovný posun: + SCREEN_WIDTH // 2. Horní okraj: + 50 px.

Zelené dlaždice

```
pygame.draw.polygon(
    screen, GREEN, points)
pygame.draw.polygon(
    screen, GRAY, points, 1)
```

Hráč uprostřed dlaždice

```
iso_x, iso_y = grid_to_iso(x, y)
center = (iso_x,
    iso_y + TILE_HEIGHT // 2)
pygame.draw.circle(
    screen, RED, center, 10)
```

Držení šipek a čekání

player_x a player_y jsou indexy polí. Krok 1 = jedna dlaždice.

Výřez: RIGHT při držení

```
keys = pygame.key.get_pressed()

if keys[pygame.K_RIGHT]:
    if player_x < GRID_WIDTH - 1:
        player_x += 1
        pygame.time.delay(100)
```

Osy mřížky

RIGHT: doprava dolů
(+32, +16) px

UP: doprava nahoru
(+32, −16) px

Čekání v hlavní smyčce

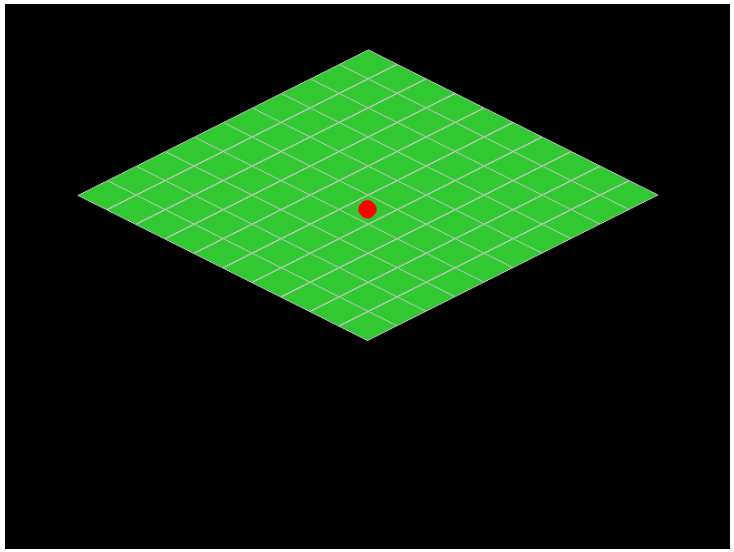
tick(60) nastavuje horní limit 60 FPS.

Během delay(100) nečteme události ani nekreslíme.

RIGHT + DOWN: dva platné směry = 2 × 100 ms čekání.

11_IsometricTiles: změna rozměrů

11_IsometricTiles/IsometricTiles.py



Výchozí mřížka, hráč na (5, 5).

Úkol v editoru

Změňte rozměry jedné dlaždice.
Počet buněk ponechte 10 × 10.

Výchozí rozměry

```
TILE_WIDTH, TILE_HEIGHT = 64, 32
```

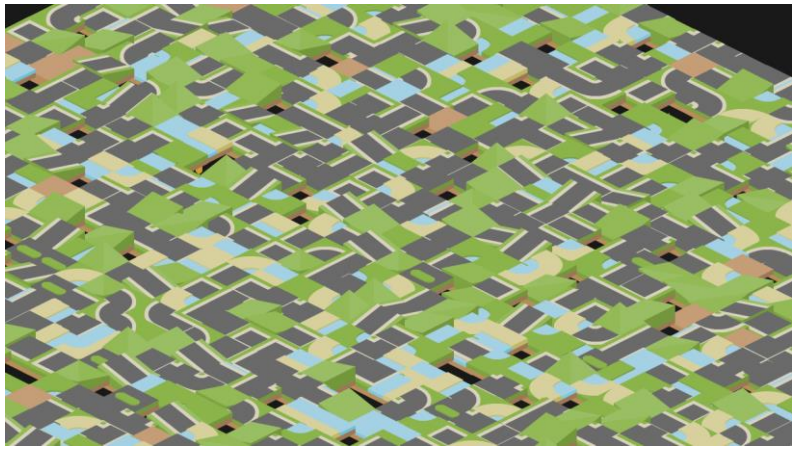
Rozměry pro cvičení

```
TILE_WIDTH, TILE_HEIGHT = 48, 24
```

Mřížka se zmenší; poměr 2:1 zůstane.

12_IsometricTiles: mapa z obrázků

12_IsometricTiles/main.py



PNG načteme jednou.
Mapa: 40 × 40 náhodných indexů.

Ovládání

Držte levé tlačítko a táhněte.
ESC nebo zavření okna: konec.

Posun mapy při tažení

Při MOUSEMOTION během tažení:

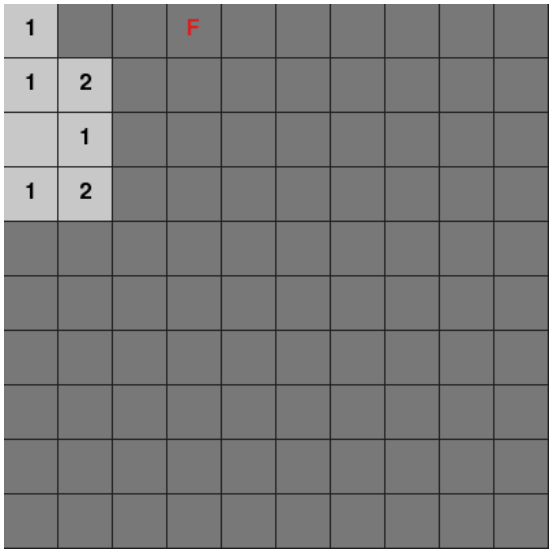
```
dx, dy = event.rel
offset_x += dx
offset_y += dy
```

Relativní pohyb myši mění oba posuny.

13_MINY: pravidla a data

13_MINY/Miny.py

10 × 10 polí, 15 min. Výhra: odkrýt všech 85 bezpečných polí.



Ovládání

Levé tlačítko: odkrýt pole.
Pravé tlačítko: přepnout vlajku.

Jedna buňka = slovník

bomb — přítomnost miny
revealed — odkryté pole
flag — vlajka
adj — počet min kolem bezpečného pole

Výhra i prohra zastaví herní klikání.

Odkrývání pomocí zásobníku

```
mx, my = event.pos
col = mx // TILE
row = my // TILE
```

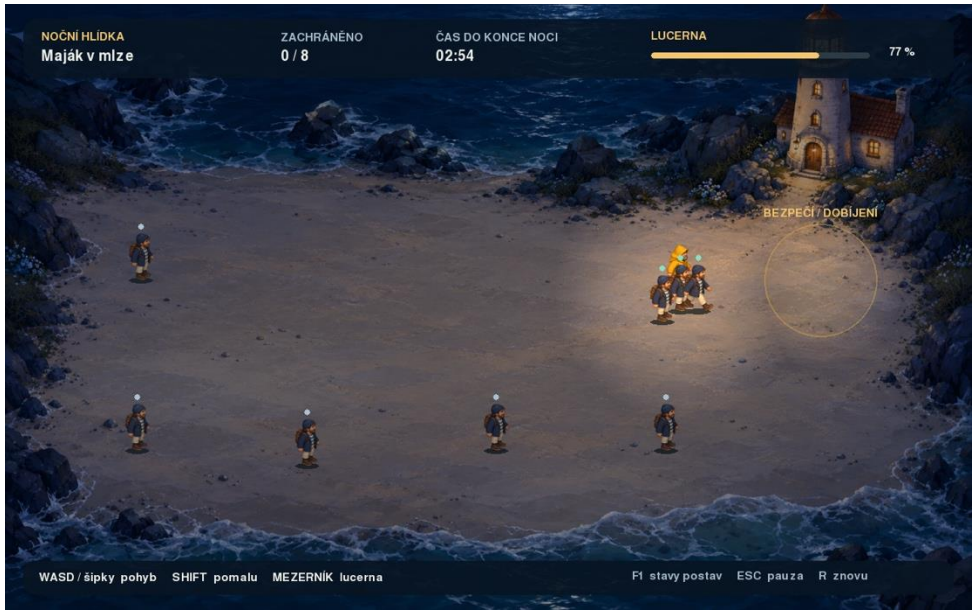
Z pixelů získáme board[row][col].
reveal() předává jen bezpečné pole.

```
def flood_fill(r, c):
    stack = [(r, c)]
    while stack:
        r, c = stack.pop()
        cell = board[r][c]
        if cell["revealed"] or cell["flag"]:
            continue
        cell["revealed"] = True
        if cell["adj"] == 0:
            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < ROWS and 0 <= nc < COLS:
                        if not board[nr][nc]["revealed"]:
                            stack.append((nr, nc))
```

Maják v mlze

18_Majak_v_mlze/main.py

Doveď 8 trosečníků do bezpečí před koncem noci.



Lucerna a energie

Ve světle tě trosečníci následují.
V bezpečí u majáku se energie dobíjí.

Známá herní smyčka

Události → stav hry
→ aktualizace → vykreslení

Stavy trosečníka

18_Majak_v_mlze/postavy.py

`ztraceny`	Stojí na místě. Světlo a vzdálenost ≤ 150 px → nasleduje.
`nasleduje`	Jde za strážcem. Při svícení vydrží až do 170 px. Zhasnutí nebo vzdálenost > 170 px → vystraseny.
`vystraseny`	Stojí asi 2 herní sekundy, potom → ztraceny. Světlo do 150 px jej vrátí hned do nasleduje.
`zachraneny`	Někdy následoval a jeho nohy jsou v kruhu bezpečí. Konečný stav: další přechody se neprovádějí.

Ozvěny času: záznam a přehrávání

19_Ozveny_casu/main.py

historie: seznam vzorků (čas, x, y, směr, snímek)

```
historie.append((cas_zaznamu, hrac.pozice.x, hrac.pozice.y,
                 hrac.smer, hrac.snimek))
```

1. Z menu otevři místnost Enterem. Mezerník pak vrátí hráče na start a zapne nahrávání.

2. Ukládáme i stání, nejvýše 8 herních sekund.
Záznam ukončí další mezerník nebo časový limit.

3. Ozvěna přehrává historii.
Hráče na startu znovu ovládáš klávesami.

Desky a otevření dveří



1. Nahraj cestu na A a na desce záznam ukonči.

2. Ozvěna dojde na A a zůstane tam. S hráčem dojdi na B.

3. A i B musí být současně sepnuté 0,6 herní sekundy. Pak můžeš z B odejít k východu.

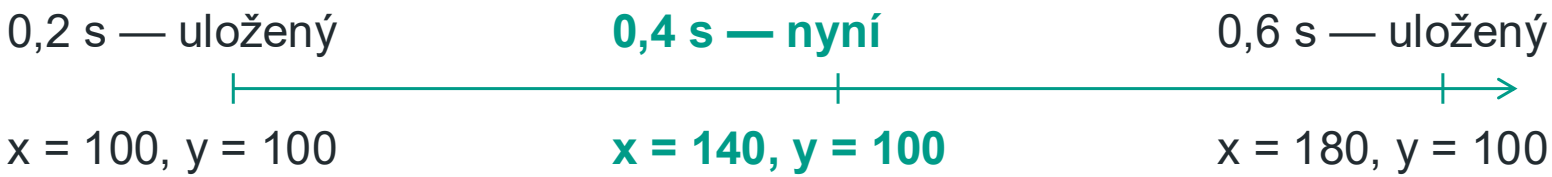
Dveře zůstanou otevřené až do resetu pokusu.

Osm herních sekund omezuje nahrávku, nikoli dobu na dokončení místnosti.

Ozvěna: poloha mezi dvěma záznamy

19_Ozveny_casu/postavy.py

Ukázková dvojice sousedních vzorků, ne skutečná nahrávka.



Podíl: (0,4 − 0,2) / (0,6 − 0,2) = 0,5

Aktuální vzorek je (cas, x, y); následuje (dalsi_cas, dalsi_x, dalsi_y).

```
self.pozice = pygame.Vector2(x, y)
podil = (self.cas - cas) / (dalsi_cas - cas)
self.pozice = self.pozice.lerp((dalsi_x, dalsi_y), podil)
```

lerp() vrací nový Vector2; podil je v rozsahu 0–1.
Interpolujeme záznam, nepočítáme znovu klávesy ani kolize.

Místnost jako data

19_Ozveny_casu/urovne.py

První místnost: 9 × 16 znaků

```
################
#..........####
#..A........####
#..........#..#
#.S........D.E#
#..........#..#
#........B..####
#..........####
################
```

Význam znaků

zeď
. podlaha
S start
A/B desky
D dveře
E východ

A: sloupec 3, řádek 2 (od nuly)

Roh dlaždice: (320, 268)
Střed: (352, 300)

Změníme mapu, herní pravidla zůstávají stejná.

AI Cart: stavba projektu

AI_Cart_engine_PUBLIC/main.py

main.py

Připraví okno, grafiku a agenty.

scenes/ a SceneManager

Přepínají menu, jízdu a trénink.

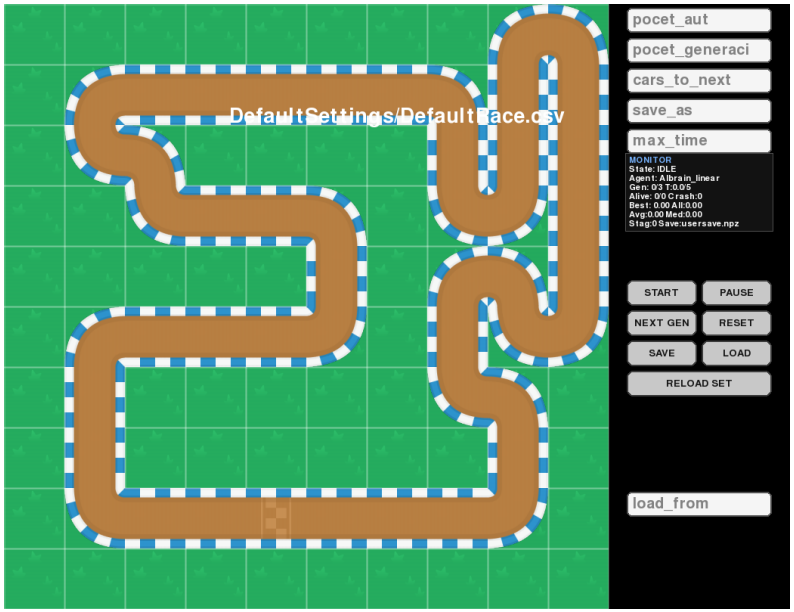
core/ a my_sprites/

Mapy, evoluce, auta a kolize.

students/

Vlastní agenti a uložené výsledky.

Student tvoří vlastní mozek auta.



Tréninková scéna před START

Jak auto rozhoduje

students/agents/AIbrain_linear.py

Vstup: 9 vzdáleností

Paprsky měří vzdálenost k okraji sjízdné oblasti, v dlaždicích.

```
z = self.W.dot(x) + self.b
```

4 × 9 + 4 = 40 parametrů

36 vah a 4 posuny

Výstup: 4 skóre pro povely

Zrychlit

Brzdit

Zatočit doleva

Zatočit doprava

Každé skóre > 0,5 zapne příslušný povel.

Více povelů může být aktivních současně.

Výběr a mutace

core/car_manager.py

1. Populace

Každé auto má vlastní mozek.

2. Hodnocení

Lineární agent: celková ujetá dráha.

3. Výběr rodičů

Nejvyšší skóre mají přednost.

4. Další generace

Kopie rodičů a jejich mutované kopie.

Ručně zadaný příklad

10 aut, 3 rodiče

3 elitní kopie

Parametry se nemění.

7 mutovaných kopií

Náhodné změny parametrů.

Zachování parametrů nezaručuje lepší příští jízdu.

Data a porovnávání agentů

AI_Cart_engine_PUBLIC/students/

.py: pravidla rozhodování

agents/

.npz: uložené parametry

saves/

.csv: mapa po dlaždicích

maps/

CSV a PNG: průběh tréninku

results/training_logs/
results/training_plots/

Trénink

Lineární agent odměňuje ujetou dráhu.

Benchmark

Dojezd do cíle a čas.
Vzdálenost je doplňkový údaj.

Porovnávání výsledků

Stejná mapa a nastavení umožňují
srovnat výsledky více běhů.

Grafika a podmínky použití

Kenney

Herní assety z asset pages mají licenci CC0.

OpenGameArt, itch.io, CraftPix

Podmínky a uvedení autora se řídí konkrétním balíčkem.

AI grafika

Může doplnit vlastní vizuální styl projektu.

Zdarma ke stažení neznamená stejná oprávnění k použití.

kenney.nl/assets opengameart.org
itch.io/game-assets/free